

Supporting Information

Cohen and Schneidman 10.1073/pnas.1211606110

SI Text

Ruling Out Simple Memorization as the Strategy That Subjects Use.

We first consider a memorization scheme in which subjects perfectly learn a list of correct pattern-label pairs but do not generalize. The acquisition of such memorized patterns can be an all-or-none event, which means that after a certain pattern-label pair was memorized it will dictate choice behavior. Alternatively, we consider a gradual probabilistic memory strengthening process for the observed patterns, which leaves room for errors.

Our data rules out the case of all-or-none memorization: Fig. S1A demonstrates that our subjects frequently made mistakes on specific patterns even after they were labeled correctly in previous blocks. We can also rule out memorization of the types mentioned above as the sole mechanism: relying on memorization alone would mean that all rules on patterns of four bits would be learned in the same rate. Fig. 1 clearly shows that this is not the case, and that rule complexity plays a key role.

Even finer memorization aspects, such as pattern-specific acquisition rates, can also be ruled out from our data. Fig. S1B shows that subjects learned the labeling of the same pattern under different rules at different rates.

We note that considering a memorization scheme with pattern-specific memorization rates that also include dependencies between memorization processes of different patterns, one ends up with a memorization process that is not very similar to our learning mixture model.

Finally, in a summary questionnaire which subjects completed after all of the learning sessions were finished, many of them reported that they found a labeling “rule,” and very few reported adopting a strategy of memorizing salient patterns. Clearly, this observation may be a post hoc generalization, but it further supports the existence of a generalization mechanism.

Dependence of Learning in the Mixture of Features Model on $\eta\beta$. The learning rule of the mixture of features model can be interpreted as a rotation of the coefficients vectors $\vec{\alpha}^c$ (because we change $\vec{\alpha}^c$ and then renormalize their total length to be 1). The angle of rotation of the coefficients’ vectors from $\vec{\alpha}_t$ to $\vec{\alpha}_{t+1}$ in a single learning step is given by

$$\begin{aligned} \cos \theta &= \vec{\alpha}_t \cdot \vec{\alpha}_{t+1} = \vec{\alpha}_t \cdot \frac{\beta \vec{\alpha}_t + \eta \frac{\partial \log L}{\partial \vec{\alpha}_t}}{\left\| \beta \vec{\alpha}_t + \eta \frac{\partial \log L}{\partial \vec{\alpha}_t} \right\|} \\ &= \frac{1 + \eta \beta \frac{\partial \log L}{\partial \vec{\alpha}_t} \cdot \vec{\alpha}_t}{\left(1 + 2 \eta \beta \vec{\alpha}_t \cdot \frac{\partial \log L}{\partial \vec{\alpha}_t} + \eta^2 \beta^2 \left\| \frac{\partial \log L}{\partial \vec{\alpha}_t} \right\|^2 \right)^{1/2}}, \end{aligned}$$

where we dropped the category notation c for clarity. Thus, the model’s learning dynamics depends to a large extent on the ratio of the learning rate and the confidence parameter, $\eta\beta$, where $\left\| \frac{\partial \log L}{\partial \vec{\alpha}_t} \right\|$ that depends on the value of β alone is bounded by $\left\| \frac{\partial \log L}{\partial \vec{\alpha}_t} \right\| = |1 - P(y|\vec{x})| \cdot |f_i(\vec{x})_{p(\vec{x}|c)} - f_i(\vec{x})| \leq 2$.

Comparisons of the Mixture of Features Model with Other Models.

We compare the accuracy of the mixture of features model in describing and predicting learning behavior to that of a neural networks model and a reinforcement learning model.

Mixture of features model. At each learning step, the probability to assign the label, $Y \in \{-1, 1\}$, to each possible pattern, $\vec{x} \in \{-1, 1\}^N$, is given by the probabilistic classifier (Eq. 2 in main text)

$$P(Y = 1 | \vec{X} = \vec{x}) = \frac{1}{1 + \sum_{\mu=-1}^1 \exp \left(\beta \sum_{\mu} f_{\mu}(\vec{x}) \cdot \alpha_{\mu} + \gamma \right)},$$

with $\vec{\alpha} = \vec{\alpha}^{-1} - \vec{\alpha}^1$ (see main text).

In this framework, learning is incrementally achieved by changing the model’s coefficients according to the gradient ascent on the log-likelihood of the most recent examples in a window, W :

$$\begin{aligned} \frac{\partial \log L}{\partial \alpha_{\nu}^1} &= -\frac{1}{|W|} \left\{ \sum_{\vec{x} \in W, y(\vec{x})=1} P(0|\vec{x}) \left(\langle f_{\nu}(\vec{x}) \rangle_{p(\vec{x}|1)} - f_{\nu}(\vec{x}) \right) \right. \\ &\quad \left. - \sum_{\vec{x} \in W, y(\vec{x})=-1} P(1|\vec{x}) \left(\langle f_{\nu}(\vec{x}) \rangle_{p(\vec{x}|1)} - f_{\nu}(\vec{x}) \right) \right\} \quad [\text{S1}] \end{aligned}$$

$$\begin{aligned} \frac{\partial \log L}{\partial \alpha_{\nu}^{-1}} &= \frac{1}{|W|} \left\{ \sum_{\vec{x} \in W, y(\vec{x})=1} P(0|\vec{x}) \left(\langle f_{\nu}(\vec{x}) \rangle_{p(\vec{x}|-1)} - f_{\nu}(\vec{x}) \right) \right. \\ &\quad \left. - \sum_{\vec{x} \in W, y(\vec{x})=-1} P(1|\vec{x}) \left(\langle f_{\nu}(\vec{x}) \rangle_{p(\vec{x}|-1)} - f_{\nu}(\vec{x}) \right) \right\} \quad [\text{S2}] \end{aligned}$$

$$\frac{\partial \log L}{\partial \gamma} = -\frac{1}{|W|} \left\{ \sum_{\vec{x} \in W, y(\vec{x})=1} P(0|\vec{x}) - \sum_{\vec{x} \in W, y(\vec{x})=-1} P(1|\vec{x}) \right\}. \quad [\text{S3}]$$

$\langle f_{\nu} \rangle_{p(\vec{x}|c)}$ is the expected value of feature f_{ν} over the model distribution given by Eq. 1 in the main text.

A scalar learning rate, η , determines the learning step, $\vec{\alpha}^c \rightarrow \vec{\alpha}^c + \eta \frac{\partial \log L}{\partial \vec{\alpha}^c}$, $\gamma \rightarrow \gamma + \eta \frac{\partial \log L}{\partial \gamma}$, which is followed by renormalizing the weight vector such that $|\vec{\alpha}^c| = 1$.

In the following section we show that using $|W| = 1$ does not impair the models’ performance compared with a general choice of W . However, this simplifies Eqs. S1 and S2.

Neural networks model. The probabilistic classifier is modeled by a three-layer perceptron model (1):

$$P(Y = 1 | \vec{X} = \vec{x}) = \frac{1}{1 + \exp(-2\beta h)}, \quad [\text{S4}]$$

where β is the slope of the sigmoid form and controls the confidence of the model, and $h = \vec{W}^T \cdot \begin{bmatrix} \vec{V} \\ 1 \end{bmatrix}$ is the output node. $\vec{W}$ are the connections weights between the hidden layer and the output node.

The hidden layer nodes are

$$\vec{V} = \tanh \left(\beta \cdot w \cdot \begin{bmatrix} \vec{x} \\ 1 \end{bmatrix} \right), \quad [\text{S5}]$$

where w is a connection weights matrix between the pattern cues ($\vec{x}$) in the input level and the hidden layer. This network is guaranteed to be able to approximate any input/output relation using a single hidden layer (2). When fitting these models to subjects’ sessions we allowed the number of hidden layer nodes to vary as long as the total number of parameters did not exceed the number

of mixture model parameters. The learning step algorithm is given by the back-propagation algorithm, which stems from gradient descent on the squared error, $E = (y(\vec{x}) - P(Y = 1|\vec{X} = \vec{x}))^2$:

$$\Delta \vec{W} = 2\eta\beta(y(\vec{x}) - P) \cdot P \cdot (1 - P) \cdot \vec{V}, \quad [S6]$$

where $y(\vec{x})$ is the correct label of $\vec{x}$, and

$$\Delta w = 2\eta\beta^2(y(\vec{x}) - P) \cdot P \cdot (1 - P) \cdot (1 - h^2) \vec{W} \cdot \vec{x}^T, \quad [S7]$$

followed by the resetting the lengths of all of the input weights vectors, $\vec{W}$, $\vec{w}_i$, to length 1.

Reinforcement learning model (adapted from ref. 3). The probabilistic classifier is given by

$$P = \frac{1}{1 + \exp\left(-\beta \cdot \sum_{\mu} \alpha_{\mu} f_{\mu}(\vec{x})\right)}, \quad [S8]$$

following the same notations as the mixture of features model.

The learning algorithm is as follows: (i) change only the coefficients that correspond to the chosen category ($\vec{\alpha}^1$ if category 1 was chosen, etc.); (ii) if the choice i was correct, then $\delta\alpha_j^i = \eta \cdot (1 - \alpha_j^i) f_j(\vec{x})$; (iii) if the choice i was incorrect, then $\delta\alpha_j^i = \eta \cdot (-\alpha_j^i) f_j(\vec{x})$.

Comparisons. Comparisons were made by fitting the three models to each session of each of the subjects. Thus, for each modeling framework we found the confidence parameter, β , learning rate, η , and the prior value of the free parameters to maximize the likelihood that each model gives to the subject's answers. Fig. S2 shows that the features mixture model performs better or at least as well as the other models in terms of the number of overlap with the subjects' answers. However, importantly, the mixture model always achieved its performance with a much better similarity Z-score to the subjects.

Effect of the Length of the Learning Window on the Mixture of Features Model. We used mixture of features models with a memory window of length 1 in the main text. Longer windows did not give significantly better-fitting results. Fig. S3 shows an example.

Pattern-Specific Performance Before Personalized Training Block, and the Order of Patterns in That Block. Fig. S4 shows examples of the learning performance on the 16 different patterns during the first three blocks (where patterns were shown in random order) and the order of presentation in the fourth (personalized training) block, which was optimized individually for each subject. As intuition may suggest, we found that in general, patterns that were labeled incorrectly by the subject appeared more in the fourth block. However, somewhat surprisingly, some of the patterns that were skipped were ones that the subject misclassified often (two of three), whereas some of the patterns that were correctly classified in all blocks were still chosen by the optimization process. We note that we did not find an obvious relation between the failure (success) rate of the patterns in the first three blocks and their position in the fourth block (see the following section for differences between learning algorithms).

Optimality of the Personalized Training Sequence: Comparison with Alternative Models. To assess the specific effect of using the mixture models to fit subjects, we also tried both the neural networks and reinforcement learning models to estimate what would be the optimal sequence of patterns in the fourth block for

the subjects in our experiment if they operated according to these models. We then simulated performance of individual subjects using the mixture models that were fitted to those subjects, but presented these models with either the mixture model sequence (MM), the sequence predicted by the neural network model (NN), or the sequence of the reinforcement learning (RL) model. Fig. S5A shows the expected difference in learning averaged over the population between the mixture-based model and the two others. We found that the mixture model predicts a small but significant advantage of using the individual block of patterns according to that model. Importantly, beyond the population average, this advantage was found for every individual subject we tested. To further reflect the difference between the sequences of patterns that these models predict to be the optimal one, Fig. S5B shows the histograms of the number of unique patterns in these personalized teaching sequences according to the different models.

Mixture Model as a Simple Case of Prototype or Exemplar Representation. We note that there is a natural way to view our mixture model as an exemplar representation of the stimulus space.

Eq. 1 in the main text, $P(\vec{x}|y = c) = \frac{1}{Z} \exp(\beta \sum_i \alpha_i^c f_i(\vec{x}))$, gives the internal model that a subject has for the probability of each pattern to belong to category c as a weighted mixture of features, $f_i(\vec{x})$, of the pattern $\vec{x}$, summed with coefficients α_i .

We can choose the coefficients vector, $\vec{a}$, to be in the span of the set of prototypes, $\{\vec{f}(\vec{x})\}_{\vec{x} \in X}$. By rewriting $\vec{a} = \sum_{\vec{x} \in X} \lambda_{\vec{x}} \vec{f}(\vec{x})$, Eq. 1 takes the shape of projections on the prototypes, as in ref. 4.

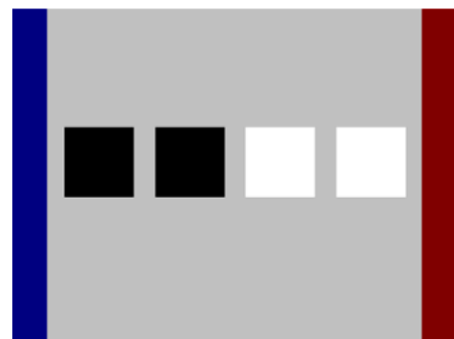
Instructions to Subjects in the Pattern Classification Task. General. As part of a scientific experiment in the neurobiology department you are asked to participate in a pattern classification task. The task is made of four separate and independent games.

Next, we describe how to play these games.

Game description. Goal. The goal is to classify patterns, made of white and black squares, into two groups with the maximal accuracy.

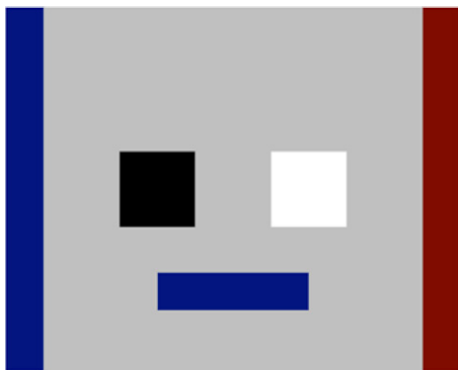
Beginning a game. Each game will start with a gray screen with the inscription "Click to begin." Pressing a mouse button will start the game.

Playing the game. In each game, patterns will be presented sequentially. Each pattern is a row of black and white squares on a gray background as shown in the next picture.



After the pattern is presented you have 10 seconds to classify it. Classification is done by pressing one of the two colored buttons (blue and red slabs in the picture above) on the sides of the screen. The buttons' location is for convenience only and their shape, color, or position is not related to the patterns. If no classification was made during the 10 seconds after pattern presentation then the inscription "Time Out, Next Pattern" will appear above the pattern and it will be replaced.

After choosing the classification (blue/red), a slab with the correct color will appear under the pattern for one second (as in the following picture) followed by the next pattern.



Finishing the game and moving to the next game. The game will end in one of the following two conditions

- After presenting 128 patterns (regardless of the answers)
- When many patterns have been classified correctly in a sequence (varies from one game to another)

When the game ends it will close. If more games remain (there are 4 in total) then a new game will begin and the inscription "Click to begin" will reappear.

Before starting the next game you must take a short break outside the room.

Important. If the same pattern appears in two different games then its classification in one game is completely unrelated to the classification in the other game.

1. Duda RO, Hart PE, Stork DG (2001) *Pattern Classification* (Wiley, New York).
2. Hornik K, Stinchcombe M, White H (1989) Multilayer feedforward networks are universal approximators. *Neural Netw* 2(5):359–366.
3. Soltani A, Lee D, Wang X-J (2006) Neural mechanism for stochastic behaviour during a competitive game. *Neural Netw* 19(8):1075–1090.

4. Kruschke JK (1992) ALCOVE: An exemplar-based connectionist model of category learning. *Psychol Rev* 99(1):22–44.

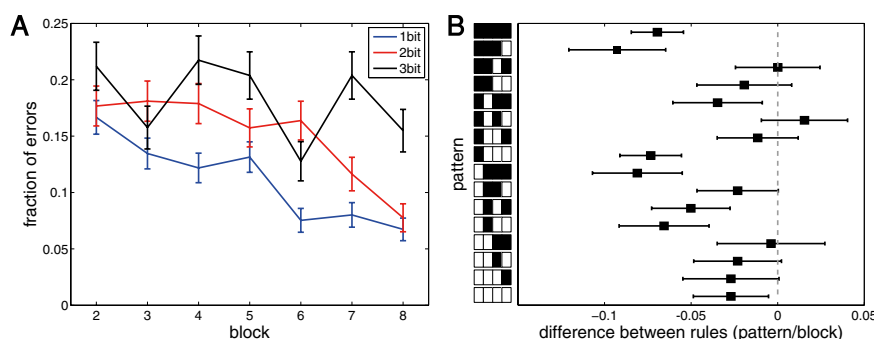


Fig. S1. Ruling out simple memorization schemes as the strategy that subjects use to learn the rules. (A) Because each pattern appeared once in each block, we asked, What is the fraction of patterns were labeled incorrectly in each block after a correct labeling in the previous block? We plot the population mean fraction of these "forgetting" occurrences. The error bars are SEM and the colors show different rules. (B) For each pattern (shown on the y-axis) we present the population average difference in forgetting rate (occurrences per block shown on x-axis) between the one-bit rule and the two-bit rule. The markers show the average over the population and the blocks; error bars show SEM.

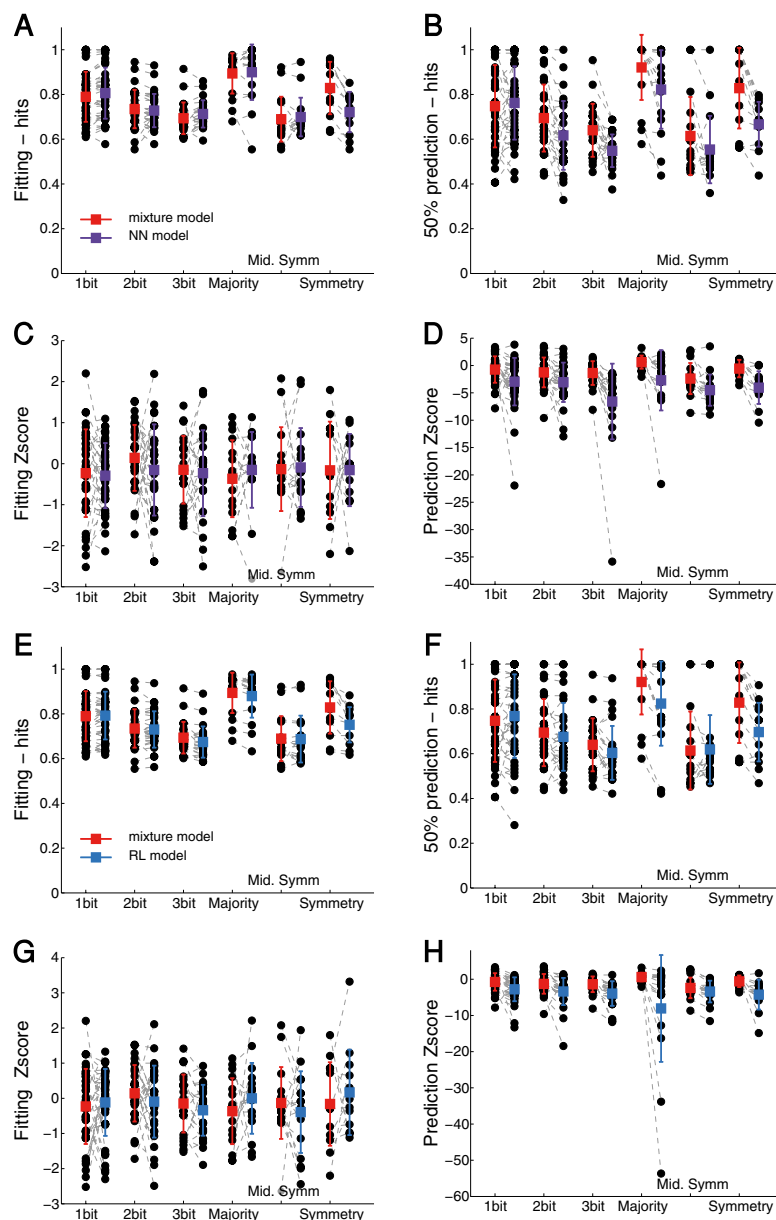


Fig. S2. Comparison of mixture of features, neural network (NN), and reinforcement learning (RL) models in describing individual learning sessions. Each session is represented by a pair of connected dots; the left dot is the mixture model results, and the right dot is the other model; this is done for each of the rules (x-axis). Red (purple, blue) error bars are the population's mean and SD of the mixture (NN, RL) model. (A) Comparison of fraction of fitted answers (y-axis) between the mixture of features model and a NN model. (B) Comparison of the Z-scores of the fits for the same models as in A. (C) Comparison of the fraction of predicted answers for the models from A. (D) Comparison of the prediction Z-scores from A. (E) Comparison of the fitted answers in the mixture of features model to the RL algorithm. (F) Comparison of the Z-scores of the fits in E. (G) Comparison of the fraction of predicted answers by the mixture of features algorithm and the RL algorithm. (H) Comparison of the predicted Z-scores from G.

